

Modern Operating Systems — Chapter 4 File Systems Exercises and Answers

Andrew S. Tanenbaum and Herbert Bos

Contents

Chapter 4: File Systems	4
Exercise 1	4
Exercise 2	4
Exercise 3	5
Exercise 4	5
Exercise 5	5
Exercise 6	6
Exercise 7	6
Exercise 8	7
Exercise 9	7
Exercise 10	7
Exercise 11	8
Exercise 12	8
Exercise 13	8
Exercise 14	9
Exercise 15	9

Exercise 16	10
Exercise 17	10
Exercise 18	10
Exercise 19	11
Exercise 20	11
Exercise 21	12
Exercise 22	12
Exercise 23	12
Exercise 24	13
Exercise 25	13
Exercise 26	14
Exercise 27	14
Exercise 28	14
Exercise 29	15
Exercise 30	15
Exercise 31	16
Exercise 32	16
Exercise 33	16
Exercise 34	17
Exercise 35	17
Exercise 36	18

Exercise 37	18
Exercise 38	18
Exercise 39	19
Exercise 40	19
Exercise 41	20
Exercise 42	20
Exercise 43	21
Exercise 44	22
Exercise 45	23
Exercise 46	24
Exercise 47	25
Exercise 48	27

Chapter 4: File Systems

Source note. Questions were extracted from the Chapter 4 problem section of *Modern Operating Systems*, 4th edition. Answers 1–42 were matched to the Chapter 4 answer section of the provided `MOS_4th_answer.pdf`. Exercises 43–48 had no matching answer in that answer section, so their answers are clearly labeled as prepared technical answers rather than answer-manual text.

Exercise 1

Question

Give five different path names for the file `/etc/passwd`. (Hint: Think about the directory entries “.” and “..”)

Answer

You can go up and down the tree as often as you want using “..”. Some of the many paths are `/etc/passwd` `./etc/passwd` `../etc/passwd` `../../etc/passwd` `/etc/../etc/passwd` `/etc/../etc/../etc/passwd` `/etc/../etc/../etc/../etc/passwd` `/etc/../etc/../etc/../etc/../etc/passwd`

Source: Answer manual.

Exercise 2

Question

In Windows, when a user double clicks on a file listed by Windows Explorer, a program is run and given that file as a parameter. List two different ways the operating system could know which program to run.

Answer

The Windows way is to use the file extension. Each extension corresponds to a file type and to some program that handles that type. Another way is to remember which program created the file and run that program. The Macintosh works this way.

Source: Answer manual.

Exercise 3

Question

In early UNIX systems, executable files (a.out files) began with a very specific magic number, not one chosen at random. These files began with a header, followed by the text and data segments. Why do you think a very specific number was chosen for executable files, whereas other file types had a more-or-less random magic number as the first word?

Answer

These systems loaded the program directly in memory and began executing at word 0, which was the magic number. To avoid trying to execute the header as code, the magic number was a BRANCH instruction with a target address just above the header. In this way it was possible to read the binary file directly into the new process' address space and run it at 0, without even knowing how big the header was.

Source: Answer manual.

Exercise 4

Question

Is the open system call in UNIX absolutely essential? What would the consequences be of not having it?

Answer

To start with, if there were no open, on every read it would be necessary to specify the name of the file to be opened. The system would then have to fetch the i-node for it, although that could be cached. One issue that quickly arises is when to flush the i-node back to disk. It could time out, however. It would be a bit clumsy, but it might work.

Source: Answer manual.

Exercise 5

Question

Systems that support sequential files always have an operation to rewind files. Do systems that support random-access files need this, too?

Answer

No. If you want to read the file again, just randomly access byte 0.

Source: Answer manual.

Exercise 6

Question

Some operating systems provide a system call `rename` to give a file a new name. Is there any difference at all between using this call to rename a file and just copying the file to a new file with the new name, followed by deleting the old one?

Answer

Yes. The `rename` call does not change the creation time or the time of last modification, but creating a new file causes it to get the current time as both the creation time and the time of last modification. Also, if the disk is nearly full, the copy might fail.

Source: Answer manual.

Exercise 7

Question

In some systems it is possible to map part of a file into memory. What restrictions must such systems impose? How is this partial mapping implemented?

Answer

The mapped portion of the file must start at a page boundary and be an integral number of pages in length. Each mapped page uses the file itself as backing store. Unmapped memory uses a scratch file or partition as backing store.

Source: Answer manual.

Exercise 8

Question

A simple operating system supports only a single directory but allows it to have arbitrarily many files with arbitrarily long file names. Can something approximating a hierarchical file system be simulated? How?

Answer

Use file names such as `/usr/ast/file`. While it looks like a hierarchical path name, it is really just a single name containing embedded slashes.

Source: Answer manual.

Exercise 9

Question

In UNIX and Windows, random access is done by having a special system call that moves the “current position” pointer associated with a file to a given byte in the file. Propose an alternative way to do random access without having this system call.

Answer

One way is to add an extra parameter to the read system call that tells what address to read from. In effect, every read then has a potential for doing a seek within the file. The disadvantages of this scheme are (1) an extra parameter in every read call, and (2) requiring the user to keep track of where the file pointer is.

Source: Answer manual.

Exercise 10

Question

Consider the directory tree of Fig. 4-8. If `/usr/jim` is the working directory, what is the absolute path name for the file whose relative path name is `../ast/x`?

Answer

The dotdot component moves the search to `/usr`, so `../ast` puts it in `/usr/ast`. Thus `../ast/x` is the same as `/usr/ast/x`.

Source: Answer manual.

Exercise 11

Question

Contiguous allocation of files leads to disk fragmentation, as mentioned in the text, because some space in the last disk block will be wasted in files whose length is not an integral number of blocks. Is this internal fragmentation or external fragmentation? Make an analogy with something discussed in the previous chapter.

Answer

Since the wasted storage is between the allocation units (files), not inside them, this is external fragmentation. It is precisely analogous to the external fragmentation of main memory that occurs with a swapping system or a system using pure segmentation.

Source: Answer manual.

Exercise 12

Question

Describe the effects of a corrupted data block for a given file for:

- (a) contiguous,
- (b) linked, and
- (c) indexed (or table based).

Answer

If a data block gets corrupted in a contiguous allocation system, then only this block is affected; the remainder of the file's blocks can be read. In the case of linked allocation, the corrupted block cannot be read; also, location data about all blocks starting from this corrupted block is lost. In case of indexed allocation, only the corrupted data block is affected.

Source: Answer manual.

Exercise 13

Question

One way to use contiguous allocation of the disk and not suffer from holes is to compact the disk every time a file is removed. Since all files are contiguous, copying a file requires a seek and rotational delay to read the file, followed by the transfer at full speed. Writing the file back requires the same work. Assuming a seek time of 5 msec, a rotational delay of 4

msec, a transfer rate of 80 MB/sec, and an average file size of 8 KB, how long does it take to read a file into main memory and then write it back to the disk at a new location? Using these numbers, how long would it take to compact half of a 16-GB disk?

Answer

It takes 9 msec to start the transfer. To read 2^{13} bytes at a transfer rate of 80 MB/sec requires 0.0977 msec, for a total of 9.0977 msec. Writing it back takes another 9.0977 msec. Thus, copying a file takes 18.1954 msec. To compact half of a 16-GB disk would involve copying 8 GB of storage, which is 2^{20} files. At 18.1954 msec per file, this takes 19,079.25 sec, which is 5.3 hours. Clearly, compacting the disk after every file removal is not a great idea.

Source: Answer manual.

Exercise 14

Question

In light of the answer to the previous question, does compacting the disk ever make any sense?

Answer

If done right, yes. While compacting, each file should be organized so that all of its blocks are consecutive, for fast access. Windows has a program that defragments and reorganizes the disk. Users are encouraged to run it periodically to improve system performance. But given how long it takes, running once a month might be a good frequency.

Source: Answer manual.

Exercise 15

Question

Some digital consumer devices need to store data, for example as files. Name a modern device that requires file storage and for which contiguous allocation would be a fine idea.

Answer

A digital still camera records some number of photographs in sequence on a nonvolatile storage medium (e.g., flash memory). When the camera is reset, the medium is emptied. Thereafter, pictures are recorded one at a time in sequence until the medium is full, at which time they are uploaded to a hard disk. For this application, a contiguous file system inside the camera (e.g., on the picture storage medium) is ideal.

Source: Answer manual.

Exercise 16

Question

Consider the i-node shown in Fig. 4-13. If it contains 10 direct addresses and these were 8 bytes each and all disk blocks were 1024 KB, what would the largest possible file be?

Answer

The indirect block can hold 128 disk addresses. Together with the 10 direct disk addresses, the maximum file has 138 blocks. Since each block is 1 KB, the largest file is 138 KB.

Source: Answer manual.

Exercise 17

Question

For a given class, the student records are stored in a file. The records are randomly accessed and updated. Assume that each student's record is of fixed size. Which of the three allocation schemes (contiguous, linked and table/indexed) will be most appropriate?

Answer

For random access, table/indexed and contiguous will be both appropriate, while linked allocation is not as it typically requires multiple disk reads for a given record.

Source: Answer manual.

Exercise 18

Question

Consider a file whose size varies between 4 KB and 4 MB during its lifetime. Which of the three allocation schemes (contiguous, linked and table/indexed) will be most appropriate?

Answer

Since the file size changes a lot, contiguous allocation will be inefficient requiring reallocation of disk space as the file grows in size and compaction of free blocks as the file

shrinks in size. Both linked and table/indexed allocation will be efficient; between the two, table/indexed allocation will be more efficient for random-access scenarios.

Source: Answer manual.

Exercise 19

Question

It has been suggested that efficiency could be improved and disk space saved by storing the data of a short file within the i-node. For the i-node of Fig. 4-13, how many bytes of data could be stored inside the i-node?

Answer

There must be a way to signal that the address-block pointers hold data, rather than pointers. If there is a bit left over somewhere among the attributes, it can be used. This leaves all nine pointers for data. If the pointers are k bytes each, the stored file could be up to 9k bytes long. If no bit is left over among the attributes, the first disk address can hold an invalid address to mark the following bytes as data rather than pointers. In that case, the maximum file is 8k bytes.

Source: Answer manual.

Exercise 20

Question

Two computer science students, Carolyn and Elinor, are having a discussion about inodes. Carolyn maintains that memories have gotten so large and so cheap that when a file is opened, it is simpler and faster just to fetch a new copy of the i-node into the inode table, rather than search the entire table to see if it is already there. Elinor disagrees. Who is right?

Answer

Elinor is right. Having two copies of the i-node in the table at the same time is a disaster, unless both are read only. The worst case is when both are being updated simultaneously. When the i-nodes are written back to the disk, whichever one gets written last will erase the changes made by the other one, and disk blocks will be lost.

Source: Answer manual.

Exercise 21

Question

Name one advantage of hard links over symbolic links and one advantage of symbolic links over hard links.

Answer

Hard links do not require any extra disk space, just a counter in the i-node to keep track of how many there are. Symbolic links need space to store the name of the file pointed to. Symbolic links can point to files on other machines, even over the Internet. Hard links are restricted to pointing to files within their own partition.

Source: Answer manual.

Exercise 22

Question

Explain how hard links and soft links differ with respect to i-node allocations.

Answer

A single i-node is pointed to by all directory entries of hard links for a given file. In the case of soft-links, a new i-node is created for the soft link and this i-node essentially points to the original file being linked.

Source: Answer manual.

Exercise 23

Question

Consider a 4-TB disk that uses 4-KB blocks and the free-list method. How many block addresses can be stored in one block?

Answer

The number of blocks on the disk = $4 \text{ TB} / 4 \text{ KB} = 2^{30}$. Thus, each block address can be 32 bits (4 bytes), the nearest power of 2. Thus, each block can store $4 \text{ KB} / 4 = 1024$ addresses.

Source: Answer manual.

Exercise 24

Question

Free disk space can be kept track of using a free list or a bitmap. Disk addresses require D bits. For a disk with B blocks, F of which are free, state the condition under which the free list uses less space than the bitmap. For D having the value 16 bits, express your answer as a percentage of the disk space that must be free.

Answer

The bitmap requires B bits. The free list requires DF bits. The free list requires fewer bits if $DF < B$. Alternatively, the free list is shorter if $F/B < 1/D$, where F/B is the fraction of blocks free. For 16-bit disk addresses, the free list is shorter if 6% or less of the disk is free.

Source: Answer manual.

Exercise 25

Question

The beginning of a free-space bitmap looks like this after the disk partition is first formatted: 1000 0000 0000 0000 (the first block is used by the root directory). The system always searches for free blocks starting at the lowest-numbered block, so after writing file A, which uses six blocks, the bitmap looks like this: 1111 1110 0000 0000. Show the bitmap after each of the following additional actions:

- (a) File B is written, using five blocks.
- (b) File A is deleted.
- (c) File C is written, using eight blocks.
- (d) File B is deleted.

Answer

The beginning of the bitmap looks like:

- (a) After writing file B: 1111 1111 1111 0000
- (b) After deleting file A: 1000 0001 1111 0000
- (c) After writing file C: 1111 1111 1111 1100
- (d) After deleting file B: 1111 1110 0000 1100

Source: Answer manual.

Exercise 26

Question

What would happen if the bitmap or free list containing the information about free disk blocks was completely lost due to a crash? Is there any way to recover from this disaster, or is it bye-bye disk? Discuss your answers for UNIX and the FAT-16 file system separately.

Answer

It is not a serious problem at all. Repair is straightforward; it just takes time. The recovery algorithm is to make a list of all the blocks in all the files and take the complement as the new free list. In UNIX this can be done by scanning all the i-nodes. In the FAT file system, the problem cannot occur because there is no free list. But even if there were, all that would have to be done to recover it is to scan the FAT looking for free entries.

Source: Answer manual.

Exercise 27

Question

Oliver Owl's night job at the university computing center is to change the tapes used for overnight data backups. While waiting for each tape to complete, he works on writing his thesis that proves Shakespeare's plays were written by extraterrestrial visitors. His text processor runs on the system being backed up since that is the only one they have. Is there a problem with this arrangement?

Answer

Ollie's thesis may not be backed up as reliably as he might wish. A backup program may pass over a file that is currently open for writing, as the state of the data in such a file may be indeterminate.

Source: Answer manual.

Exercise 28

Question

We discussed making incremental dumps in some detail in the text. In Windows it is easy to tell when to dump a file because every file has an archive bit. This bit is missing in UNIX. How do UNIX backup programs know which files to dump?

Answer

They must keep track of the time of the last dump in a file on disk. At every dump, an entry is appended to this file. At dump time, the file is read and the time of the last entry noted. Any file changed since that time is dumped.

Source: Answer manual.

Exercise 29

Question

Suppose that file 21 in Fig. 4-25 was not modified since the last dump. In what way would the four bitmaps of Fig. 4-26 be different?

Answer

In

(a) and (b), 21 would not be marked. In (c), there would be no change. In (d), 21 would not be marked.

Source: Answer manual.

Exercise 30

Question

It has been suggested that the first part of each UNIX file be kept in the same disk block as its i-node. What good would this do?

Answer

Many UNIX files are short. If the entire file fits in the same block as the inode, only one disk access would be needed to read the file, instead of two, as is presently the case. Even for longer files there would be a gain, since one fewer disk accesses would be needed.

Source: Answer manual.

Exercise 31

Question

Consider Fig. 4-27. Is it possible that for some particular block number the counters in both lists have the value 2? How should this problem be corrected?

Answer

It should not happen, but due to a bug somewhere it could happen. It means that some block occurs in two files and also twice in the free list. The first step in repairing the error is to remove both copies from the free list. Next a free block has to be acquired and the contents of the sick block copied there. Finally, the occurrence of the block in one of the files should be changed to refer to the newly acquired copy of the block. At this point the system is once again consistent.

Source: Answer manual.

Exercise 32

Question

The performance of a file system depends upon the cache hit rate (fraction of blocks found in the cache). If it takes 1 msec to satisfy a request from the cache, but 40 msec to satisfy a request if a disk read is needed, give a formula for the mean time required to satisfy a request if the hit rate is h . Plot this function for values of h varying from 0 to 1.0.

Answer

The time needed is $h + 40 \times (1 - h)$. The plot is just a straight line.

Source: Answer manual.

Exercise 33

Question

For an external USB hard drive attached to a computer, which is more suitable: a writethrough cache or a block cache?

Answer

In this case, it is better to use a write-through cache since it writes data to the hard drive while also updating the cache. This will ensure that the updated file is always on the

external hard drive even if the user accidentally removes the hard drive before disk sync is completed.

Source: Answer manual.

Exercise 34

Question

Consider an application where students' records are stored in a file. The application takes a student ID as input and subsequently reads, updates, and writes the corresponding student record; this is repeated till the application quits. Would the "block readahead" technique be useful here?

Answer

The block read-ahead technique reads blocks sequentially, ahead of their use, in order to improve performance. In this application, the records will likely not be accessed sequentially since the user can input any student ID at a given instant. Thus, the read-ahead technique will not be very useful in this scenario.

Source: Answer manual.

Exercise 35

Question

Consider a disk that has 10 data blocks starting from block 14 through 23. Let there be 2 files on the disk: f1 and f2. The directory structure lists that the first data blocks of f1 and f2 are respectively 22 and 16. Given the FAT table entries as below, what are the data blocks allotted to f1 and f2? (14,18); (15,17); (16,23); (17,21); (18,20); (19,15); (20, -1); (21, -1); (22,19); (23,14). In the above notation, (x, y) indicates that the value stored in table entry x points to data block y.

Answer

The blocks allotted to f1 are: 22, 19, 15, 17, 21. The blocks allotted to f2 are: 16, 23, 14, 18, 20.

Source: Answer manual.

Exercise 36

Question

Consider the idea behind Fig. 4-21, but now for a disk with a mean seek time of 6 msec, a rotational rate of 15,000 rpm, and 1,048,576 bytes per track. What are the data rates for block sizes of 1 KB, 2 KB, and 4 KB, respectively?

Answer

At 15,000 rpm, the disk takes 4 msec to go around once. The average access time (in msec) to read k bytes is then $6 + 2 + (k/1,048,576) \times 4$. For blocks of 1 KB, 2 KB, and 4 KB, the access times are about 6.0039 msec, 6.0078 msec, and 6.0156 msec, respectively (hardly any different). These give rates of about 170.556 KB/sec, 340.890 KB/sec, and 680.896 KB/sec, respectively.

Source: Answer manual.

Exercise 37

Question

A certain file system uses 4-KB disk blocks. The median file size is 1 KB. If all files were exactly 1 KB, what fraction of the disk space would be wasted? Do you think the wastage for a real file system will be higher than this number or lower than it? Explain your answer.

Answer

If all files were 1 KB, then each 4-KB block would contain one file and 3 KB of wasted space. Trying to put two files in a block is not allowed because the unit used to keep track of data is the block, not the semiblock. This leads to 75% wasted space. In practice, every file system has large files as well as many small ones, and these files use the disk much more efficiently. For example, a 32,769-byte file would use 9 disk blocks for storage, given a space efficiency of $32,769/36,864$, which is about 89%.

Source: Answer manual.

Exercise 38

Question

Given a disk-block size of 4 KB and block-pointer address value of 4 bytes, what is the largest file size (in bytes) that can be accessed using 10 direct addresses and one indirect block?

Answer

The indirect block can hold 1024 addresses. Added to the 10 direct addresses, there are 1034 addresses in all. Since each one points to a 4-KB disk block, the largest file is 4,235,264 bytes

Source: Answer manual.

Exercise 39

Question

Files in MS-DOS have to compete for space in the FAT-16 table in memory. If one file uses k entries, that is k entries that are not available to any other file, what constraint does this place on the total length of all files combined?

Answer

It constrains the sum of all the file lengths to being no larger than the disk. This is not a very serious constraint. If the files were collectively larger than the disk, there would be no place to store all of them on the disk.

Source: Answer manual.

Exercise 40

Question

A UNIX file system has 4-KB blocks and 4-byte disk addresses. What is the maximum file size if i-nodes contain 10 direct entries, and one single, double, and triple indirect entry each?

Answer

The i-node holds 10 pointers. The single indirect block holds 1024 pointers. The double indirect block is good for 1024^2 pointers. The triple indirect block is good for 1024^3 pointers. Adding these up, we get a maximum file size of 1,074,791,434 blocks, which is about 16.06 GB.

Source: Answer manual.

Exercise 41

Question

How many disk operations are needed to fetch the i-node for a file with the path name /usr/ast/courses/os/handout.t? Assume that the i-node for the root directory is in memory, but nothing else along the path is in memory. Also assume that all directories fit in one disk block.

Answer

The following disk reads are needed: directory for / i-node for /usr directory for /usr i-node for /usr/ast directory for /usr/ast i-node for /usr/ast/courses directory for /usr/ast/courses i-node for /usr/ast/courses/os directory for /usr/ast/courses/os i-node for /usr/ast/courses/os/handout.t In total, 10 disk reads are required.

Source: Answer manual.

Exercise 42

Question

In many UNIX systems, the i-nodes are kept at the start of the disk. An alternative design is to allocate an i-node when a file is created and put the i-node at the start of the first block of the file. Discuss the pros and cons of this alternative.

Answer

Some pros are as follows. First, no disk space is wasted on unused i-nodes. Second, it is not possible to run out of i-nodes. Third, less disk movement is needed since the i-node and the initial data can be read in one operation. Some cons are as follows. First, directory entries will now need a 32-bit disk address instead of a 16-bit i-node number. Second, an entire disk will be used even for files which contain no data (empty files, device files). Third, file-system integrity checks will be slower because of the need to read an entire block for each i-node and because i-nodes will be scattered all over the disk. Fourth, files whose size has been carefully designed to fit the block size will no longer fit the block size due to the i-node, messing up performance.

Source: Answer manual.

Exercise 43

Question

Write a program that reverses the bytes of a file, so that the last byte is now first and the first byte is now last. It must work with an arbitrarily long file, but try to make it reasonably efficient.

Answer

A safe implementation should write the reversed data to a new output file instead of modifying the input file in place. It reads fixed-size chunks from the end of the input, reverses each chunk in memory, and appends it to the output. This works for arbitrarily long files because memory use is bounded by the buffer size.

```
#!/usr/bin/env python3
import os
import sys

BUF = 1024 * 1024

def reverse_file(src, dst):
    if os.path.abspath(src) == os.path.abspath(dst):
        raise SystemExit("refusing to overwrite the input file in place")
    size = os.path.getsize(src)
    with open(src, "rb") as inp, open(dst, "wb") as out:
        pos = size
        while pos > 0:
            n = min(BUF, pos)
            pos -= n
            inp.seek(pos)
            out.write(inp.read(n)[::-1])

if __name__ == "__main__":
    if len(sys.argv) != 3:
        raise SystemExit("usage: reverse_file.py input output")
    reverse_file(sys.argv[1], sys.argv[2])
```

Run it as:

```
python reverse_file.py input.bin reversed.bin
python reverse_file.py reversed.bin restored.bin
```

Expected behavior: `reversed.bin` starts with the last byte of `input.bin`; running the program a second time restores the original order. Safety: keep the output on a file system with enough free space, and do not use the same path for input and output.

Source: Independently prepared technical answer.

Exercise 44

Question

Write a program that starts at a given directory and descends the file tree from that point recording the sizes of all the files it finds. When it is all done, it should print a histogram of the file sizes using a bin width specified as a parameter (e.g., with 1024, file sizes of 0 to 1023 go in one bin, 1024 to 2047 go in the next bin, etc.).

Answer

One simple solution is to walk the directory tree recursively, count only regular files, divide each file size by the requested bin width, and print the counts.

```
#!/usr/bin/env python3
import os
import sys
from collections import defaultdict

def histogram(root, width):
    bins = defaultdict(int)
    for dirpath, dirnames, filenames in os.walk(root):
        for name in filenames:
            path = os.path.join(dirpath, name)
            try:
                st = os.stat(path, follow_symlinks=False)
            except OSError:
                continue
            if not os.path.isfile(path):
                continue
            bins[st.st_size // width] += 1
    return bins

if __name__ == "__main__":
    if len(sys.argv) != 3:
        raise SystemExit("usage: sizehist.py directory bin_width")
    width = int(sys.argv[2])
    for b, count in sorted(histogram(sys.argv[1], width).items()):
        print(f"{b * width:10d}-{(b + 1) * width - 1:10d}: {count}")
```

Run it as:

```
python sizehist.py /path/to/tree 1024
```

Expected behavior: files of sizes 0–1023 appear in the first bin, 1024–2047 in the next bin, and so on. Safety: run it first on a small test directory; on a large tree it may take a long time and may encounter permission errors, which the program skips.

Source: Independently prepared technical answer.

Exercise 45

Question

Write a program that scans all directories in a UNIX file system and finds and locates all i-nodes with a hard link count of two or more. For each such file, it lists together all file names that point to the file.

Answer

Hard links to the same file have the same device number and i-node number. A scan can group path names by `(st_dev, st_ino)` and print groups whose link count is two or more.

```
#!/usr/bin/env python3
import os
import sys
from collections import defaultdict

def find_hard_links(root):
    groups = defaultdict(list)
    for dirpath, dirnames, filenames in os.walk(root):
        for name in filenames:
            path = os.path.join(dirpath, name)
            try:
                st = os.lstat(path)
            except OSError:
                continue
            if st.st_nlink >= 2 and os.path.isfile(path):
                groups[(st.st_dev, st.st_ino)].append(path)
    for (dev, ino), paths in sorted(groups.items()):
        if len(paths) >= 2:
            print(f"dev={dev} inode={ino}")
            for p in sorted(paths):
                print(f" {p}")

if __name__ == "__main__":
```

```
root = sys.argv[1] if len(sys.argv) > 1 else "."
find_hard_links(root)
```

Run it as:

```
python hardlinks.py /path/to/filesystem
```

Expected behavior: each printed group lists path names that refer to the same underlying file. Safety: scanning an entire mounted file system can be slow; start with a small subtree and avoid crossing into pseudo-file systems such as `/proc`.

Source: Independently prepared technical answer.

Exercise 46

Question

Write a new version of the UNIX `ls` program. This version takes as an argument one or more directory names and for each directory lists all the files in that directory, one line per file. Each field should be formatted in a reasonable way given its type. List only the first disk address, if any.

Answer

A portable `ls`-like program can print metadata available from `stat()`: mode, link count, owner, group, size, modification time, and name. The first physical disk address is not portable across file systems; on Linux, an approximate first extent can be obtained with `filefrag -v` when the file system supports extents.

```
#!/usr/bin/env python3
import os
import pwd
import grp
import stat
import sys
import time

def list_dir(path):
    with os.scandir(path) as entries:
        for e in sorted(entries, key=lambda x: x.name):
            try:
                st = e.stat(follow_symlinks=False)
            except OSError as err:
```

```

        print(f"{e.name}: {err}", file=sys.stderr)
        continue
    mode = stat.filemode(st.st_mode)
    user = pwd.getpwuid(st.st_uid).pw_name
    group = grp.getgrgid(st.st_gid).gr_name
    mtime = time.strftime("%Y-%m-%d %H:%M",
time.localtime(st.st_mtime))
    print(f"{mode} {st.st_nlink:3d} {user:8s} {group:8s} "
          f"{st.st_size:10d} {mtime} inode={st.st_ino} {e.name}")

if __name__ == "__main__":
    for d in sys.argv[1:] or ["."]:
        print(f"{d}:")
        list_dir(d)

```

Run it as:

```

python myls.py .
filefrag -v some_regular_file

```

Expected behavior: the Python program lists one line per file. The optional **filefrag** command can show the first extent or block address on supported Linux file systems. Safety: do not treat physical block numbers as stable; copy-on-write, compression, encryption, and network file systems can make them meaningless or unavailable.

Source: Independently prepared technical answer.

Exercise 47

Question

Implement a program to measure the impact of application-level buffer sizes on read time. This involves writing to and reading from a large file (say, 2 GB). Vary the application buffer size (say, from 64 bytes to 4 KB). Use timing measurement routines (such as `gettimeofday` and `getitimer` on UNIX) to measure the time taken for different buffer sizes. Analyze the results and report your findings: does buffer size make a difference to the overall write time and per-write time?

Answer

The experiment should create or use a large file, read it repeatedly with different application-level buffer sizes, and measure elapsed time. The following script uses `os.read()` so the application buffer size is explicit.

```

#!/usr/bin/env python3
import os
import sys
import time

SIZES = [64, 128, 256, 512, 1024, 2048, 4096]

def read_time(path, bufsize):
    total = 0
    start = time.perf_counter()
    fd = os.open(path, os.O_RDONLY)
    try:
        while True:
            data = os.read(fd, bufsize)
            if not data:
                break
            total += len(data)
    finally:
        os.close(fd)
    return total, time.perf_counter() - start

if __name__ == "__main__":
    if len(sys.argv) != 2:
        raise SystemExit("usage:  buftime.py largefile")
    for size in SIZES:
        total, sec = read_time(sys.argv[1], size)
        print(f"{size:5d} bytes:  {sec:8.4f} sec, {total / max(sec,
1e-9):.0f} B/s")

```

Commands for a disposable test file:

```

dd if=/dev/zero of=test-2g.bin bs=1M count=2048
python buftime.py test-2g.bin
rm test-2g.bin

```

Expected behavior: very small buffers usually perform poorly because they cause many system calls. Performance normally improves as the buffer grows and then levels off; beyond a few KB or MB, the device, cache, and file-system behavior dominate. Safety: the **dd** command creates a 2-GB file; use a smaller **count** if disk space is limited, and run the test on a noncritical disk or inside a VM. Do not claim results unless the experiment is actually run on the target system.

Source: Independently prepared technical answer.

Exercise 48

Question

Implement a simulated file system that will be fully contained in a single regular file stored on the disk. This disk file will contain directories, i-nodes, free-block information, file data blocks, etc. Choose appropriate algorithms for maintaining free-block information and for allocating data blocks (contiguous, indexed, linked). Your program will accept system commands from the user to create/delete directories, create/delete/open files, read/write from/to a selected file, and to list directory contents.

Answer

A reasonable simulated file system can use one host file as a virtual disk. The virtual disk contains a superblock, a free-block bitmap, an i-node table, directory entries, and data blocks. A simple design is indexed allocation: every file i-node stores a list of block numbers.

Minimal command set:

```
format <diskfile> <blocks>
mkdir <path>
rmdir <path>
create <path>
write <path> <text>
read <path>
rm <path>
ls <path>
```

A compact implementation strategy is:

```
BLOCK_SIZE = 4096

superblock = {
    "magic": "TINYFS1",
    "block_size": BLOCK_SIZE,
    "free": [0, 1, 0, 0, ...],
    "inodes": {
        "1": {"type": "dir", "entries": {".": 1, "..": 1}},
        "2": {"type": "file", "size": 0, "blocks": []}
    }
}

def allocate_block(fs):
    for i, used in enumerate(fs["free"]):
        if not used:
```

```

        fs["free"][i] = 1
        return i
    raise RuntimeError("disk full")

def free_block(fs, block):
    fs["free"][block] = 0

def create_file(fs, parent_inode, name):
    ino = str(max(map(int, fs["inodes"].keys())) + 1)
    fs["inodes"][ino] = {"type": "file", "size": 0, "blocks": []}
    fs["inodes"][parent_inode]["entries"][name] = int(ino)
    return ino

```

Expected behavior: **format** initializes the metadata; **mkdir** creates a directory i-node with **.** and **..**; **create** allocates a file i-node; **write** allocates data blocks and records them in the file i-node; **rm** frees the file's data blocks and i-node; **ls** prints directory entries. Safety: store the simulated disk in an ordinary file such as **tinyfs.img**, never on a raw disk device. Keep a backup copy while debugging because a bug in the simulator can corrupt the virtual disk file. This is a simulator, so it should not be mounted as a real operating-system file system.

Source: Independently prepared technical answer.